IIC3103 - Taller de Integración
Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

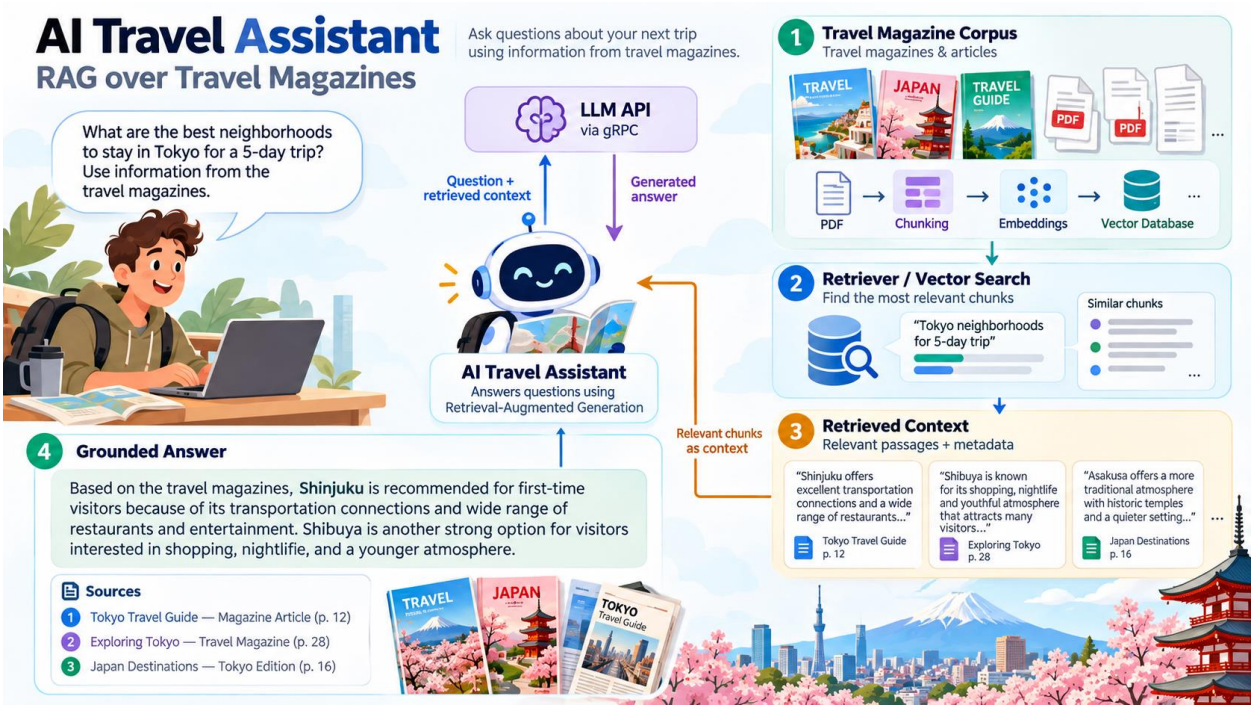
Enunciado Tarea 2

Objetivo

En esta tarea, incorporaremos la capacidad de leer bases de conocimiento y responder consultas en base a documentos usando Retrieval Augmented Generation (RAG).

Durante el proceso, aprenderás a:

- Procesar archivos pdf (Cleaning, Chunking)
- Vectorizar texto para poder indexarlo en una base vectorial (Embeddings)
- Enriquecer preguntas para mejorar resultados usando el LLM
- Consultar una base de datos vectorial para obtener fragmentos de temas similares
- Armar una respuesta usando estos fragmentos y mostrar las referencias.





IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

Objetivo 1
Trabajo a realizar 2
Contexto 2
Servicio rag_search_magazine 2
Trabajo a realizar 2
Partir de Tarea 2 2
Revistas 2
Parseo de Archivos PDF 3
Tamaño del chunk 4
Modelo tokenizer 5
Dataset propio en Nomic 5
Explorar Atlas antes del agente 6
Tool de búsqueda 7
Bibliotecas permitidas y prohibidas 8
Permitido 8
Prohibido 8
Versionamiento de código 9
Entregables 9
Fecha de entrega 9
Requisitos mínimos 9
Penalizaciones 9
Uso de IA 9
Versiones del documento 10
Anexos 11
Documentación 12
Rúbrica Tentativa 12

Trabajo a realizar

Contexto

Eres estudiante de **IIC3103 – Taller de Integración**, durante el semestre 2026-2. De pronto, se te ocurre una idea que podría cambiar tu futuro para siempre: emprender y crear **IntegraTrip**, un servicio para ayudar a las personas a planificar las vacaciones de sus sueños.

Además de poder reservar vuelos, hoteles, y poder ver el clima, el bot debería ser capaz de responder dudas sobre los destinos de vacaciones, usando información verificada y fidedigna.

Para esto, decides incorporar un sistema tipo RAG para poder responder a las consultas de tus usuarios usando una base de conocimiento.

Servicio rag_search_magazine

Para incorporar esta funcionalidad al agente, incorporaremos una tool llamada rag_search_magazine a nuestras tools. Esta tool funcionará de manera especial comparado con el resto de tools, en el sentido que no se conectará realmente con un MCP, sino que ejecutará código directamente en el backend de tu aplicación.

Trabajo a realizar

1. Bajar las 21 revistas.
2. Parsear los archivos pdf y dejar chunks con revista, páginas y encabezados.
3. Crear un dataset Nomic propio, subir los chunks e indexar el texto con gte-multilingual-base.
4. Abrir el mapa en Atlas y responder, en el README, las preguntas de exploración.
5. Agregar la tool *rag_search_magazines* al catálogo que ya recibe Generate.
6. En el loop, ejecutar esa tool en el BFF. El resto de las tools sigue yendo a tools/call del MCP que corresponda.
7. Anclar la respuesta a los fragmentos, con cita de revista y páginas, o con la frase de rechazo.

Partir de Tarea 2

Deberían poder reutilizar todo lo de la tarea 2, agregando este nuevo comportamiento de forma incremental. Es decir, se espera que se mantenga la misma interfaz de chat de la Tarea 2.

Revistas

Como base de conocimiento usaremos las revistas en español de Traveling Magazine:

https://revistatraveling.com/product-category/revistas-digitales/



Deberán elegir al menos 10 ediciones para la tarea, y anotar la selección en el README y en la entrega de la tarea.

Para descargar, generalmente las urls de las revistas siguen este formato:

https://www.tienda-revistatraveling.es/descargas/travelingNN.pdf

Por lo que pueden crear un script que descargue las ediciones que hayan elegido (revisar que estén disponibles ya que algunas están caídas.)

No deben subir los archivos PDF al repositorio.

Parseo de Archivos PDF

Cada revista debe quedar como documento estructurado y luego en chunks listos para indexar. Cada chunk que suban lleva al menos:

- número de revista
- páginas
- encabezados de la sección, si Docling los detecta
- texto del chunk

En formato jsonl.

Para esto se recomienda usar la librería docling de Python, ya que ya está validada que puede cumplir con estos requerimientos.:

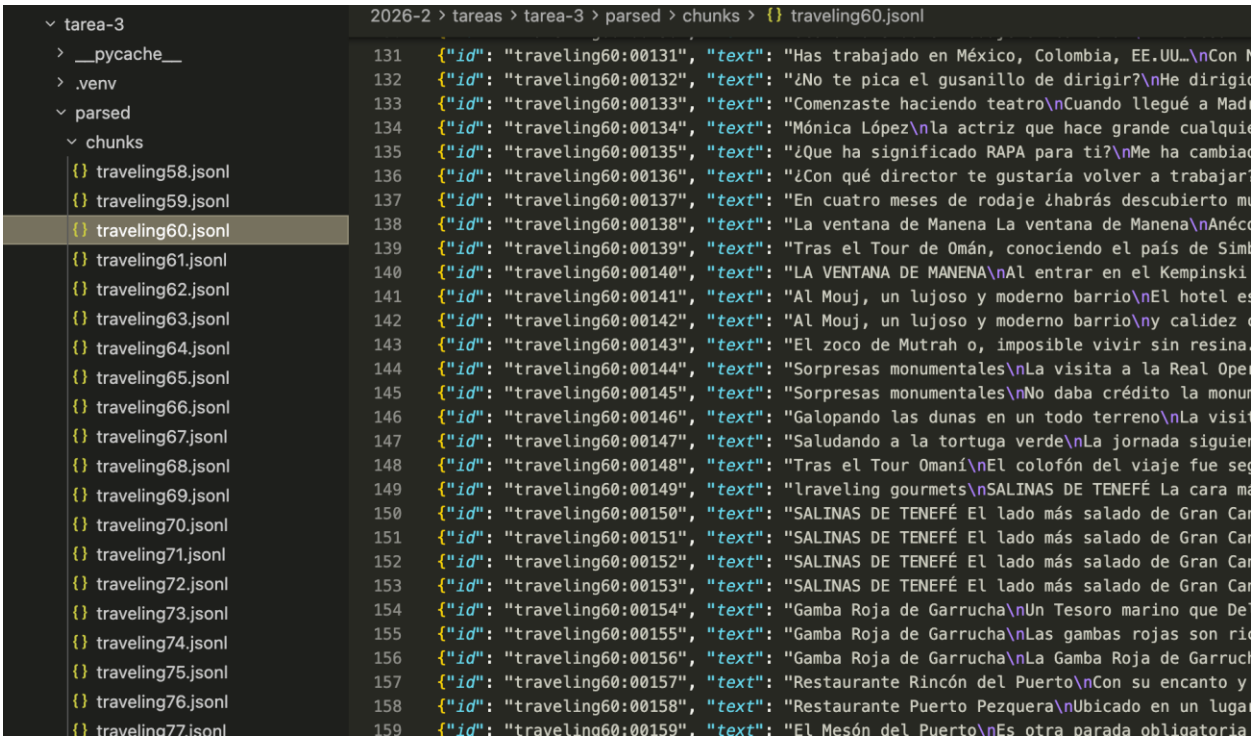
https://docling.ai/

Los PDF traen capa de texto: no activen OCR. Usen el chunker híbrido de Docling. No suban la imagen de la página ni el binario del PDF a Nomic.

El código de docling debería verse similar a esto:

```
converter = DocumentConverter()
result = converter.convert(pdf_path)
document = result.document
chunks = HybridChunker(
    tokenizer=...
).chunk(document)
```

Los archivos procesados (en formato jsonl) sí deben incluirlos en el repositorio. Ejemplo:



Algunos parámetros recomendados:

Tamaño del chunk

Chunk size	Cuándo usarlo
512	Información muy granular, FAQs, textos cortos
~700–800	⭐ Buen balance general para RAG
1.000	Artículos más narrativos y con contexto
1.500+	Cuando necesitas mantener contexto largo, pero baja la precisión de retrieval

Modelo tokenizer

sentence-transformers/all-MiniLM-L6-v2

Dataset propio en Nomic

Como base de datos vectorial usaremos Nomic Atlas:

https://atlas.nomic.ai/

Cuya capa gratuita debería bastar ampliamente para esta tarea. Deberán crear una cuenta y obtener una API Key.

Luego usando la API de Nomic, deberá generar el dataset a partir de los chunks, vectorizando el texto usando el modelo: `--embedding-model gte-multilingual-base`.

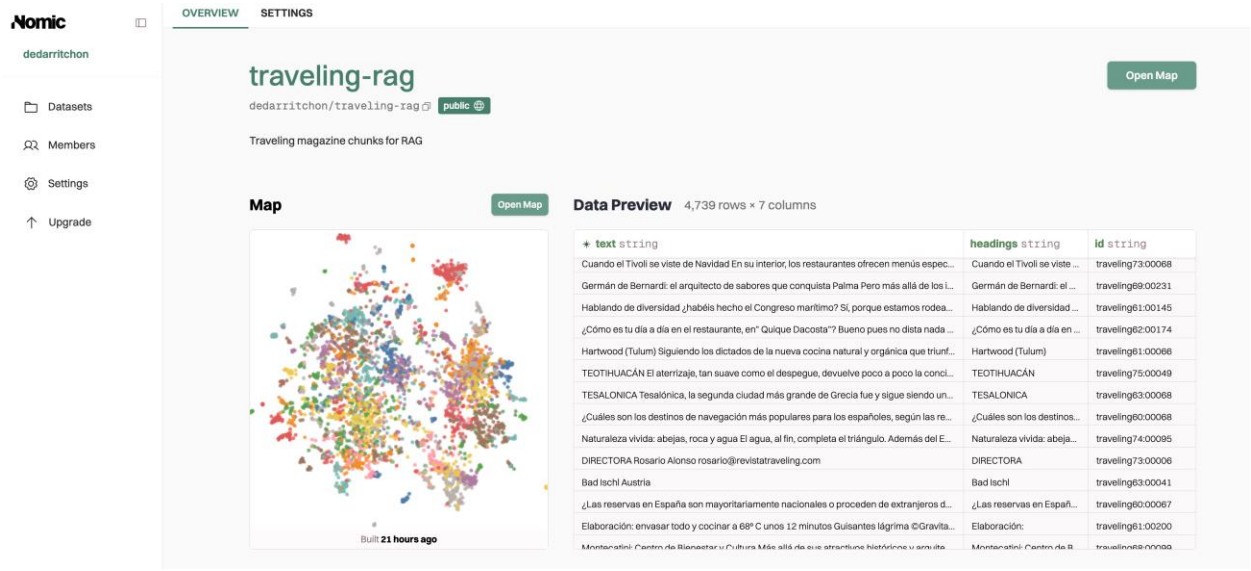
A grandes rasgos, el código para esto debería verse así:

```
rows = records()
dataset = open_dataset(args)
for start in range(0, len(rows), BATCH_SIZE):
       batch = rows[start : start + BATCH_SIZE]
       dataset.add_data(batch)
       print(f"uploaded {start + len(batch)}/{len(rows)}")
dataset.create_index(
       indexed_field="text",
       embedding_model=args.embedding_model,
)
print(f"dataset {dataset.identifier} id={dataset.id} points={dataset.total_datums}")
```

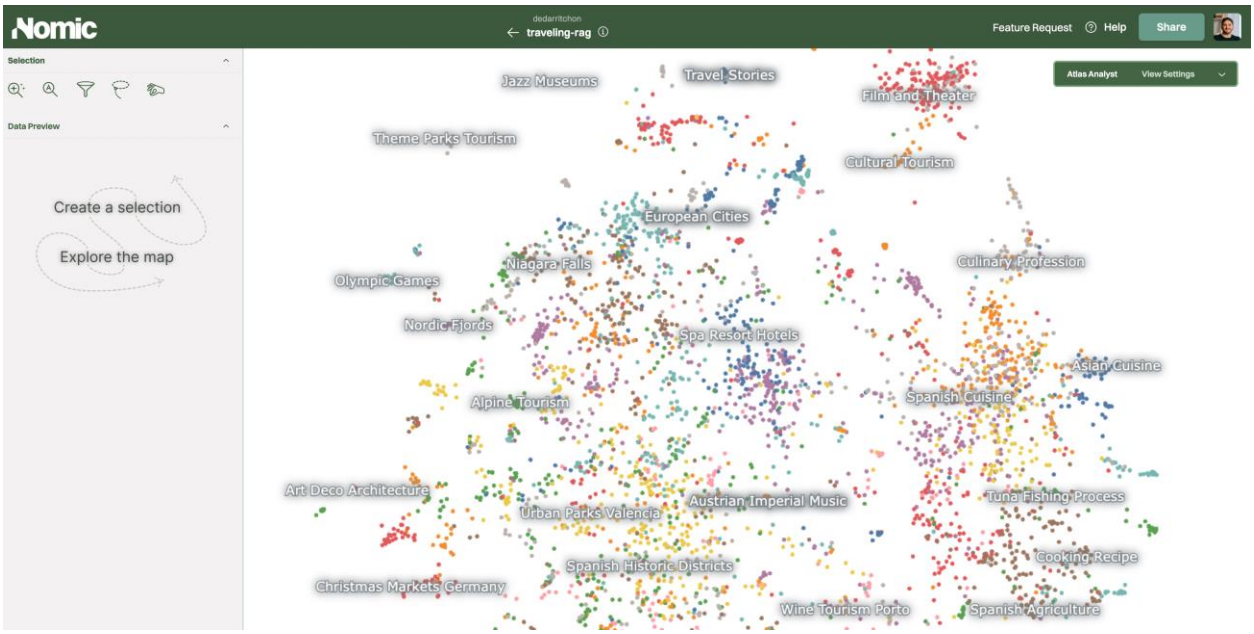
Donde rows contiene las columnas:

- Id: Identifiador único del chunk
- Text: Texto original del chunk
- Source: nomrbe de la revista ej. traveling78.pdf
- Issue: número de la revistar ej. 78
- Headings: Encabezado de la sección de donde se sacó el texto.
- Pages: Páginas de donde se sacó el texto. Ej. 4,5

Al crear su dataset, deberían poder verlo en Nomic de la siguiente forma:



Al abrir el mapa, podrán verlo con mayor detalle:



En el README incluyan la url y el identificador del dataset (organización y nombre).

No incluyan su API Key de Nomic en el README ni lo expongan en el repositorio.

Explorar Atlas antes del agente

Antes de conectar la tool, abran el mapa del dataset en Atlas. En el README, respondan con lo que ven en su mapa:

- ¿Qué temas aparecen?
- ¿Qué destinos aparecen?
- ¿Hay clusters?
- ¿Hay chunks duplicados?
- ¿Hay chunks irrelevantes?
- ¿Qué pasa si buscas "playas"?
- ¿Qué pasa si buscas "castillos"?
- ¿Qué pasa si buscas "comida romántica"?

Deberán incluir las respuestas a estas preguntas en un archivo PDF en la entrega.

Tool de búsqueda

Nombre fijo: `rag_search_magazines`.

Ahora deberán incorporar la tool a su agente que le dará la capacidad de responder en base a la información de las revistas.

Al conversar con el agente, el LLM intentará usar la tool para responder una pregunta del usuario. Esto depende altamente de la descripción que le den a la tool para que el LM pueda decidir cuándo usarla.

Una vez tenemos la necesidad de usar la tool, lo primero que haremos será pedirle al LLM que limpie la pregunta y la optimice para la búsqueda. Esto puede requerir descomponer la pregunta en varias preguntas, por ejemplo, para una pregunta compuesta:

¿Dónde puedo comer muy bien y ver atardeceres hermosos al mismo tiempo?

Si tratamos de responder usando una sola búsqueda puede funcionar mediocremente.

Mejor:

- **Q1:** restaurantes con vistas al atardecer
- **Q2:** miradores para ver el atardecer con restaurantes
- **Q3:** restaurantes románticos junto al mar para ver el atardecer

Después:

- `retrieve(Q1, k=5)`
- `retrieve(Q2, k=5)`
- `retrieve(Q3, k=5)`

y unir resultados. Esto aumentará las posibilidades de que encontremos fragmentos relacionados a la pregunta original.

En resumen:

1. El BFF, para cada query:
2. La embebe con el mismo modelo del índice (`gte-multilingual-base`, tarea de búsqueda).
3. Pide vecinos en su dataset.
4. Devuelve como máximo 4 fragmentos en total, sin duplicar el mismo chunk, con revista, páginas, encabezados y texto recortado.

En el hilo se ve la traza, igual que cualquier otra tool: nombre, argumentos y fragmentos devueltos. La respuesta final del agente cita revista y páginas.

Si el modelo pide esta tool y otras MCP en el mismo turno, ejecuten todas antes del siguiente *Generate*.

Anclaje y rechazo

El agente no debe inventar datos o responder información que no provenga de las revistas. Si los fragmentos no sirven para responder la pregunta de manera razonable, la respuesta debe ser exactamente:

"No encontré información suficiente en las revistas proporcionadas."

Si el agente inventa información, se considerará como una respuesta errónea.

Bibliotecas permitidas y prohibidas

Permitido

Lo ya permitido en Tarea 1 y Tarea 2

Docling para parsear los PDF

El cliente nomic, o HTTP a la API de Atlas, para crear el dataset, indexar y buscar

El mismo cliente gRPC de Tarea 2

Prohibido

LangChain, LlamaIndex, CrewAI, Google ADK, OpenAI Agents SDK, u otro framework que arme el loop o el RAG por ustedes

Llamar a Gemini, OpenAI u otro LLM distinto del proxy del curso, también para descomponer la pregunta o redactar la respuesta

Entregar un índice que ustedes no hayan construido desde estos PDF

Poner la API key de Nomic en el repo, en el cliente o en Generate

Mandar access tokens MCP al LLM

Se revisa el código: Docling en el parseo, el alta del dataset, y el loop (Generate → tool de Nomic o tools/call → Generate).

Versionamiento de código

Todo el código de su servicio deberá estar versionado en un repositorio privado que será provisto por el equipo docente. Para desarrollar puedes trabajar en un repositorio propio mientras no hayamos dado este acceso al repositorio final.

Entregables

Cada estudiante deberá entregar:

- URL de su servicio desplegado y disponible públicamente en internet.
- Url de dataset/mapa público.
- Documento de análisis de Dataset.
- URl de su repositorio en github.

Esta información se deberá enviar antes de la fecha de entrega a través del buzón que se habilitará en Canvas.

Fecha de entrega

La tarea debe ser entregada a más tardar el día martes 3 de Noviembre a las 18:00, y deberá quedar desplegada y disponible durante el periodo de corrección (2 semanas).

Requisitos mínimos

Las tareas que no cumplan con las siguientes condiciones no serán corregidas y serán evaluadas con la nota mínima:

- El código deberá estar versionado en su totalidad en el repositorio de GitHub Classroom de esta tarea.
- El servicio debe reflejar fielmente el código entregado en el repositorio.
- El servicio debe tener persistencia de los datos al menos durante el periodo de corrección.

Penalizaciones

Se descontarán 0,2 puntos de la nota de la tarea por cada hora de atraso en la entrega, contados a partir de la fecha estipulada en el punto anterior.

En caso de no poder desplegar la API, se evaluará con nota máxima 5,0.

Cualquier intento de copia, plagio o acto deshonesto en el desarrollo de la tarea, será penalizado con nota 1,1 de acuerdo con la política de integridad académica del DCC.

Se descontarán 0,5 puntos de la nota ante cualquier falta grave de seguridad, detalladas en la sección Seguridad de credenciales y secretos

Uso de IA

Se permite el uso de IA y agentes para la generación del código requerido en esta tarea. Se deberá versionar en el repositorio de la tarea, todas las conversaciones sostenidas con el agente de forma de validar que el desarrollo realizado es un proyecto original y no copia.

Que el estudiante no entienda lo que entrega o que su entrega incluya información falsa (alucinada) será considerado como una falta a la integridad académica.

No se permite el uso de IA en la generación del documento de arquitectura de solución requerido en esta tarea.



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

Versiones del documento

- **Versión 1:** 5/10/2026
    - Publicación del enunciado

Anexos

Documentación

- Qué es OAuth https://auth0.com/es/intro-to-iam/what-is-oauth-2
- Protocolo MCP https://modelcontextprotocol.info
- Documentación CIMD: https://client.dev/

Rúbrica Tentativa

Criterio	Descripción	Ponderación	No cumple	Bueno	Excelente
Despliegue de app		10%			
Informe de aprendizajes		10 %			
Vista de Chats		10 %			
Loop de agente		20 %			
Manejo de contexto entre conversaciones		20 %			
Visualización de mensajesy tools		10 %			
Respuestas tienen sentido y el objetivo se logra		20 %			